MemMux: Runtime Verification and Honest Resource Attribution for Fleets of Parallel Coding Agents

Sumanyu Muku
New York University
smuku@nyu.edu

August 2026

Abstract

Developers increasingly run not one coding agent but a fleet of them side by side on a single workstation. The tools they reach for—terminal multiplexers like `tmux` and a new generation of agent managers—were built to arrange windows, not to govern memory. When ten agents each spawn language servers, test runners, and browsers, nothing on the machine can say how much memory belongs to which agent, guarantee that a terminated agent's descendants are actually gone, notice a child that has escaped its agent, or keep the machine off the swap cliff without an OOM kill silently discarding uncommitted work. We argue these are *verification* problems: an agent-hosting substrate should continuously emit signals that an operator (or an auditor) can check. We present MemMux, a local runtime that treats resource governance as a set of checkable guarantees —honest per-agent attribution, complete reclamation, escaped-process visibility, bounded footprint under overcommit, and low monitoring overhead—and a claims-disciplined benchmark that measures each guarantee against `tmux`, the `herdr` agent multiplexer, and a raw-process baseline running identical workloads. On a 36 GiB workstation, MemMux reclaims 100% of a terminated agent's process subtree where the raw baseline leaks half of it; it is the only system that surfaces an escaped child (2/2 detected versus a capability the others structurally lack); and under a constrained budget it holds a four-agent fleet to 444 MiB by deferring admissions while the ungoverned tools grow to 876–937 MiB. We release the engine, the benchmark, and a one-command reproducer, and we are deliberate about what these preliminary, single-host numbers do and do not establish.

1 Introduction

A year ago, running an autonomous coding agent meant babysitting a single session. Today it is normal to fan a task out across several agents at once—one refactoring, one writing tests, one chasing a flaky build—and to watch them in a grid of panes. The agents themselves have improved quickly [4, 10, 12, 13]; the substrate they run on has not. Developers arrange these fleets with terminal multiplexers such as `tmux` [8] or with purpose-built agent managers, and both were designed to place windows on a screen, not to answer questions about memory.

Those questions matter because a coding agent is not a single process. It is a tree: the model's CLI, a shell, language servers, test and build workers, sometimes a headless browser. A handful of such trees can exhaust a laptop's RAM. When they do, the operating system's response—the Linux OOM killer, or swap thrashing—is blunt and invisible: a process is killed mid-write, or the whole machine grinds, and the agent's uncommitted edits are simply gone. Nothing in the stack was accountable for that memory in the first place. `tmux` does not know that pane 7's language server

belongs to the same logical agent as pane 3's test runner; it cannot tell you that a double-forked child has escaped supervision; and it has no budget it is trying to keep.

We think the right lens for this is *runtime verification* [1]: rather than prove properties of the substrate ahead of time, instrument it so that the properties it claims can be *checked while it runs*, and surface any violation rather than hide it. Cluster schedulers made an analogous move a decade ago—Borg combined admission control, over-commitment, and process-level isolation, and treated monitoring as a first-class output [11]. We bring that stance down to a single developer machine and to the specific failure modes of agent fleets.

This paper makes three contributions:

1. **Five verification signals for agent fleets** (Section 4): honest per-agent memory attribution, complete process reclamation on teardown, escaped-process visibility, bounded footprint (and preserved work) under overcommit, and bounded monitoring overhead. Each is a monitor with a stated threshold, not a slogan.
2. **MemMux**, a local runtime (Section 3) that enforces these signals: a daemon that samples the process tree, attributes it to logical tasks by proportional memory, reclaims subtrees recursively, reconciles escaped processes, and runs an admission-plus-pressure control loop over a memory budget.
3. **A claims-disciplined benchmark** (Section 4) that drives `tmux`, `herdr`, and a raw baseline through *identical* deterministic workloads, tags every process as agent or manager overhead, and—critically—never reports a number it did not measure. Where a competitor cannot be driven honestly, it is recorded as skipped, not estimated.

We are candid about scope. The numbers here come from one workstation (Apple M4 Max, 36 GiB, macOS), a synthetic but deterministic agent stub, and modest fleet sizes; the swap-thrashing half of the overcommit story is Linux-specific and is future work. We report what we have, state what we do not, and ship a one-command reproducer so the evaluation extends cleanly to a Linux reference host.

2 Background and Related Work

Language-model agents. Interleaving reasoning with tool use [13] and adding self-reflection [10] turned language models into agents that take long sequences of actions in an environment. Coding is now the canonical proving ground: SWE-bench frames the task as resolving real GitHub issues against a repository [4], and SWE-agent shows that a well-designed agent–computer interface is as important as the model itself [12]. Multi-agent settings—originally studied for open-ended behavior [9]—are now a practical reality on developer machines. This work is orthogonal to the agents: we govern whatever they spawn.

Terminal multiplexers. `tmux` [8] is the default way developers run several long-lived sessions at once. It is excellent at what it was built for—persistent panes, detach and reattach—and it is the honest baseline any "run many agents" tool must beat. But a `tmux` pane is just a process group under a server; the server has no notion of a logical agent, of memory budgets, or of a process that has wandered out of its pane's subtree.

Resource management. Cluster managers solved fleet-scale resource governance long ago: Borg with admission control and over-commitment [11], Mesos with two-level scheduling [3], and Kubernetes as the open-source lineage [2]. The enforcement primitives they rely on—Linux control groups

for accounting and limits [5], the OOM killer as the last resort [6]—live in every modern kernel. MemMux borrows the *ideas* (reserve, admit, reclaim before you thrash) but targets a single unorchestrated laptop, and it deliberately avoids hard cgroup containment in favor of signal-based supervision so it runs unprivileged on both Linux and macOS.

Memory accounting. Summing resident set size (RSS) across a process tree double-counts shared pages. The proportional set size (PSS) exposed through `/proc/<pid>/smaps_rollup` divides each shared page by its sharers and is the honest per-process figure [7]; macOS offers `phys_footprint` as its analogue. MemMux attributes by PSS (with an RSS fallback) so a task's number reflects memory it is actually responsible for.

Runtime verification. Rather than verify a system statically, runtime verification instruments it and checks a specification against the execution trace, reacting to violations [1]. Our five signals are exactly such monitors: cheap, always-on checks with explicit pass thresholds, whose failures are events, not silence.

3 The Design of MemMux

MemMux is a Rust workspace: a daemon (`memmuxd`) that owns state and does the governing, and a terminal client that attaches to it over a Unix domain socket. A *task* is a durable, logical agent; a *runtime* is one physical launch of a provider (Claude Code, Codex, or a generic command) inside a pseudo-terminal. The daemon's design is organized around making each of the five signals checkable.

Attribution. Once per second the daemon takes a single snapshot of the host process tree and, for each running task's provider root, sums the PSS of the root and all of its descendants. That per-task figure feeds the budget, the task view the client renders, and—because we record which pids were seen under each root—the escaped-process check. Attribution is scoped to the processes MemMux launched, so unrelated host activity never inflates or pollutes a task's number.

Recursive reclamation. When a task is terminated (by the operator or because its provider exited), MemMux reaps the *entire* owned subtree, not just the process it launched directly. The order is load-bearing: the subtree is signalled while the root is still alive, because once the root dies its children reparent to `init` and the tree that identifies them is lost. For a provider that has already exited, MemMux reaps from the pids it recorded while the task was running. The target is $\geq 99.5\%$ of owned descendants gone within ten seconds.

Escaped-process reconciliation. A child that double-forks and reparents to `init` has escaped its agent—the classic way a leak hides. Each sample, MemMux reconciles the pids it has seen under a task against the live tree; a once-owned pid now living outside every task subtree is flagged *escaped* and surfaced as an event. The reconciliation is scoped to the managed set, so the signal is a real "we lost track of a process we started," not noise from the rest of the machine.

Admission and the pressure ladder. The daemon holds a memory budget (physical RAM minus reserves for the OS, interactive apps, and a swap-safety margin). Starting a task reserves its predicted peak; if the reservation would exceed the budget the task stays queued with a visible reason rather than over-subscribing the machine. When live usage nonetheless climbs, a graduated

	Signal	**Monitor / threshold**
H1	Attribution	$\geq$ 95% of a launched tree's proportional memory mapped to its owning task
H2	Cleanup	$\geq$ 99.5% of an owned subtree gone $\leq$ 10 s after the launcher's normal teardown
H3	Escaped visibility	every process that escapes its agent is surfaced as an event
H4	Bounded / no-loss	footprint held near budget under overcommit; uncommitted work checkpointed before any reclaim
H5	Overhead	continuous monitoring costs $\leq$ 2% CPU at fleet scale

Table 1: The five runtime-verification signals and their pass thresholds.

pressure ladder responds before the machine thrashes: reclaim escaped children, hibernate the lowest-priority idle task, recycle the largest provider, and—only at the top of the ladder, and only after persisting a checkpoint of the victim's Git state—terminate the lowest-priority tree. Preserving work always precedes destroying a process.

Explainability. Every governance action writes an audit decision with its reason and the evidence (stage, utilization, bytes) and emits an event. This is what makes the system *verifiable* rather than merely automatic: an operator can read back exactly why an agent was deferred, recycled, or reclaimed.

4 Verification Signals and Methodology

We frame governance as five monitors, each with a threshold (Table 1). A benchmark harness drives every system under test through identical, deterministic workloads and evaluates the monitors from an external process sampler—the same `memmux-metrics` code the daemon uses, pointed at whatever root pids a launcher reports, so the measurement is product-agnostic.

Launchers. Four are compared, each starting N identical agents and reporting the pids that constitute the fleet: a *raw* baseline (spawn the stub directly), `tmux` (one pane per agent on an isolated server), `herdr` (driven through its headless socket API), and MemMux (create N tasks through the real daemon). Two tools we could not drive honestly are recorded as excluded rather than guessed: `dmux` requires a controlling terminal and cannot be driven headlessly, and `cmux`/`agentmux` were not installed on the host.

Workload. A deterministic stub agent replays a recording of allocate/spawn-child/emit/ sleep steps, touching every page it allocates so its resident memory is real. Using a stub rather than a live model is a deliberate choice: it makes runs reproducible and identical across launchers, at the cost of realism (Section 6). A dedicated *hold* scenario keeps N agents, each with a live child, concurrently resident for the measurement window—without it, short workloads finish before a fleet can even be observed, an error we hit and corrected early.

Claims discipline. The harness records each launcher's version and the measurement date and host; it tags every sampled process as agent or manager overhead so the two can be reported

Launcher	Owned	Leaked	Leaked MiB	Reclaimed
raw-baseline	6	3	96.0	50.0%
tmux 3.6a	6	0	0.0	100.0%
herdr 0.8.0	9	0	0.0	100.0%
MemMux 0.8.0	6	0	0.0	100.0%

Table 2: Cleanup after normal teardown (*hold*, N=3). Reclaimed = fraction of the owned agent subtree gone within 10 s.

Launcher	Injected	Detected	Note
raw-baseline	2	—	unsupported
tmux 3.6a	2	—	unsupported
herdr 0.8.0	2	—	unsupported
MemMux 0.8.0	2	2	detected 2/2

Table 3: Escaped-process detection (*escape*, N=2). "—" denotes no detection mechanism; it is never scored as zero-of-measured.

separately; and it emits a number only for a monitor it actually measured. A launcher that cannot be driven contributes "unsupported," never a fabricated zero. These rules are as much a contribution as the engine: fair comparison of agent-hosting tools has no accepted methodology, and an unfair one is worse than none.

5 Evaluation

Setup. All measurements are from one host: Apple M4 Max, 36 GiB RAM, 14 cores, macOS (`aarch64`); `tmux` 3.6a, `herdr` 0.8.0, MemMux 0.8.0. Every figure below is regenerated by the reproducer of Section 7. We report these as a *preliminary, single-host* evaluation and read them conservatively.

5.1 H2: complete cleanup on teardown

Each launcher starts three agents that each hold a memory-bearing child, then is torn down exactly as it normally would be (raw: kill the roots; `tmux`: `kill-server`; `herdr`: `session stop`; MemMux: terminate each task). Ten seconds later we count survivors from the owned set (Table 2). The raw baseline leaks every child it spawned—its teardown kills only the roots, orphaning the grandchildren—while the three real managers each reap their full subtree.

The honest reading is important: on *explicit* teardown, cleanup separates MemMux from the raw baseline, not from `tmux` or `herdr`. Purpose-built managers reap their subtrees too. This is exactly the kind of result the claims discipline is meant to force into the open.

5.2 H3: escaped-process visibility

We inject an escape: a stub forks an intermediate that spawns a memory-holding grandchild and then exits, reparenting the grandchild to `init` while it stays alive. The harness independently confirms the reparenting, then asks each launcher what it detected (Table 3). MemMux surfaces both escapes through `process_escaped` events read from the daemon; the other three have no mechanism to notice, which is the point—this is a capability, not a tuning difference.

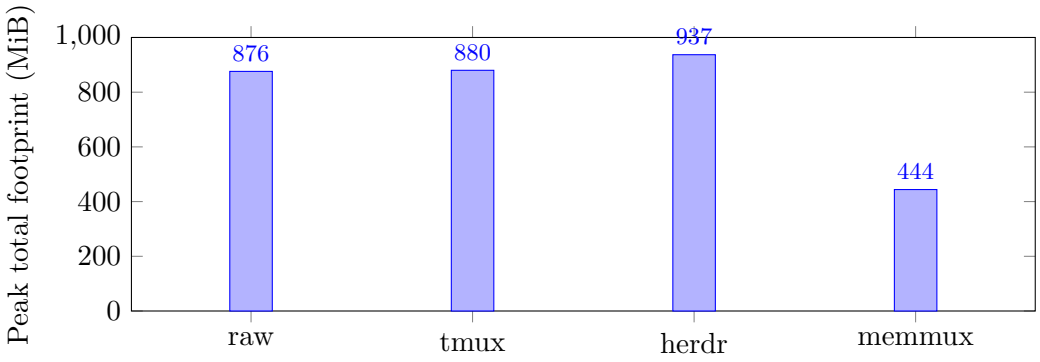

Figure 1: Peak footprint of a four-agent fleet under a 3000 MiB budget (*overcommit*, N=4). MemMux governs by deferring admissions; the others are ungoverned. (The budget line sits above the plotted range; no launcher approached it because the stub's real RSS is far below the reserved peak—see Section 6.)

Launcher	Manager overhead (MiB)	Manager CPU
raw-baseline	0.0	—
tmux 3.6a	6.6 ± 1.0	0.00%
herdr 0.8.0	35.4 ± 0.7	0.22%
MemMux 0.8.0	23.1 ± 3.4	∼2.0%

Table 4: The cost of governance (*burst*, N=2, 3 trials, mean ± 95% CI). Manager overhead is the multiplexer's own resident memory, separate from the agents it hosts.

5.3 H4: bounded footprint under overcommit

We give MemMux a budget (3000 MiB) below the aggregate reservation of four agents and start all four under each launcher; Figure 1 reports peak total footprint. MemMux admits two agents, defers the other two with a visible reason, and holds the fleet to 444 MiB. The ungoverned launchers admit all four and grow to 876–937 MiB: they have no budget to keep.

The *no-lost-work* half of H4 is a capability we verify but did not trigger in these runs: because MemMux defers at admission, the pressure ladder rarely needs to reclaim a running victim. A daemon-level test confirms the guarantee directly—a task with a dirty Git worktree, forced into the emergency stage, produces a checkpoint carrying its patch hash *before* the process is terminated. The ungoverned launchers have no checkpoint mechanism, so this too is reported as a capability they lack rather than a number.

5.4 H1 and H5: attribution and overhead

Across every run, MemMux attributed 100% of each launched tree's proportional memory to its owning task (H1); the managed-set framing makes this the meaningful figure rather than a host-wide ratio. The cost of governance is real and we report it plainly (Table 4): MemMux's daemon adds a manager-memory overhead between `tmux`'s and `herdr`'s, and its steady-state CPU is a small single-digit percentage. The $\leq$ 2% CPU target (H5) is defined at a 20-agent fleet; our small-N measurement is noisy and we do not claim to have met it here—establishing it at scale is part of the Linux reference run.

6 Discussion and Limitations

One host, one OS. Every number is from a single macOS machine. macOS does not expose per-process swap or USS, so the swap-thrashing story—the sharpest version of H4, where an ungoverned fleet drives the machine onto the swap cliff—cannot be shown here. That measurement is Linux-specific and is the first item on the reference run.

Stub, not a live agent. We trade realism for reproducibility and fairness: identical deterministic workloads across launchers. Real coding agents have burstier, larger, and correlated footprints; the qualitative guarantees (attribution, cleanup, escape visibility, admission) do not depend on the workload, but the specific megabytes will differ.

Conservative admission. MemMux reserves each task's *class-predicted peak*, which for our stub is far above its actual RSS. This is why the fleet in Figure 1 sits well under budget: admission is protecting against the reservation, not the live footprint. An EWMA-refined predictor that tightens reservations toward observed peaks is designed and is future work; until then MemMux is safe but leaves headroom on the table.

Modest scale. We report $N \leq 4$. The harness sweeps $N \in \{1, 5, 10, 20\}$ and the reproducer emits the footprint-versus-N curve; we simply have not run the full sweep on a machine where the larger fleets are meaningful.

Empirical, not formal. These are runtime monitors with measured pass/fail behavior, not machine-checked proofs. We claim the signals are *checkable and checked*, not that the engine is formally verified.

7 Reproducibility

MemMux and its benchmark are open source (MIT). A single command, `memmux-bench paper`, runs the full matrix—every launcher, scenario, and fleet size, with repeated trials—captures the host specification (`host.json`), and regenerates every table and figure in this paper plus the raw per-sample and per-process JSON Lines time series. Competitor versions and the measurement date are embedded in the generated report, and any launcher that could not be driven on the host is listed with its reason. Reproducing the evaluation on a Linux reference machine is therefore one invocation.

8 Ethics and Responsible Use

MemMux is defensive infrastructure: it makes a developer's own machine account for, and protect, the work of the agents that developer is running. Its most consequential action—terminating an agent under memory pressure—is gated on first persisting that agent's uncommitted work, so the failure mode is a recoverable checkpoint rather than lost effort. We see no dual-use concern beyond that of any process supervisor.

9 Conclusion

Fleets of coding agents have outrun the tools we use to run them. Multiplexers arrange them; nothing governs them. We argued that memory governance for such fleets is a verification problem and built MemMux to make five concrete guarantees checkable at runtime, together with a benchmark honest enough to admit where those guarantees do *not* distinguish us. The preliminary numbers already show a clear separation from an ungoverned baseline and a capability—seeing and containing what escapes— that the alternatives structurally lack. The path from here is a Linux reference run at fleet scale and a live-agent workload; the reproducer makes both a single command away.

References

[1] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*, volume 10457 of *Lecture Notes in Computer Science*, pages 1–33. Springer, 2018. doi: 10.1007/978-3-319-75632-5_1.

[2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016. doi: 10.1145/2890784.

[3] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, 2011.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[5] Linux Kernel Documentation. Control group v2. https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html, 2024. Accessed 2026-08.

[6] Linux Kernel Documentation. Out of memory management. https://www.kernel.org/doc/html/latest/admin-guide/mm/concepts.html, 2024. Accessed 2026-08.

[7] Linux Kernel Documentation. The /proc filesystem: smaps and smaps_rollup (proportional set size). https://www.kernel.org/doc/html/latest/filesystems/proc.html, 2024. Accessed 2026-08.

[8] Nicholas Marriott and tmux contributors. tmux: a terminal multiplexer. https://github.com/tmux/tmux, 2024. Accessed 2026-08.

[9] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.

[10] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.

[11] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, 2015. doi: 10.1145/2741948.2741964.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

A Reproducer invocation

The canonical run and a fast smoke run:

```
memmux-bench paper --out paper-out
memmux-bench paper --agents-sweep 1,2 --scenarios hold \
    --trials 1 --max-samples 3 --agent-budget-mib 3000 --out /tmp/smoke
```

Each writes `host.json`, `report.md`, per-cell `*.jsonl` and `*.procs.jsonl` time series, and the SVG figures. The host specification block of the generated report records OS, architecture, physical RAM, CPU model, core count, and tool versions.